

Reviewer Pack — Algebraic K-Theory and DPLL Search Tree Diameter

Entry c6cc8034e7cc · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 20:13:38 UTC

Algebraic K-Theory and DPLL Search Tree Diameter

Entry ID: c6cc8034e7cc **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-31 20:13:38 UTC

1. Conjecture

Field A (mathematical branch): Algebraic K-Theory **Field B** (complexity object): DPLL Search Trees

Statement:

The diameter of the DPLL search tree for a SAT instance is upper-bounded by the logarithm of the rank of the associated algebraic K-group, i.e., $d(\pi) \leq \text{logrank}(K_\pi)$, where π is the Tseitin representation of the SAT instance.

Rationale (proposer's reasoning):

Algebraic K-theory provides an algebraic framework to study topological invariants. The conjecture posits that the structure of DPLL search trees, which are central to understanding resolution proof complexity, can be characterized by an algebraic invariant. This bridge may reveal deeper connections between algebraic topology and computational complexity.

Taxonomy category: algebraic_k_theory_dpll_search_tree (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 36855936ec10e962

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between the diameter of the DPLL search tree ($d(\pi)$) and the logarithm of the rank of the algebraic K-group ($\text{logrank}(K_\pi)$) across 30 seeds is ≥ 0.7 , and no seed has $d(\pi) > \text{logrank}(K_\pi)$. The conjecture is falsified if any seed produces a metric where $d(\pi) > \text{logrank}(K_\pi)$, or the correlation coefficient < 0.7 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Algebraic K-Theory" AND "DPLL search tree diameter" - "Tseitin representation" AND "algebraic K-group" - "SAT instance" AND "logrank(K_n)"

Top relevant hits considered: - [s2:2407.21003] Hamiltonian elements in algebraic K-theory - [s2:10.3390/e27050531] Higher Algebraic K-Theory of Causality

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_sat_instance(n):
        clauses = []
        for i in range(1, n + 1):
            literals = [f'x{i}', f'~x{i}']
            clause = random.choice(literals)
            if random.choice([True, False]):
                clause += ' or '
            else:
                clause += ' and '
            clauses.append(clause[:-4])
        return '(' + ' and '.join(clauses) + ')'

    def tseitin_representation(sat_instance):
        literals = set()
        for literal in sat_instance.replace(' or ', '|').replace(' and ', '&').split():
            if literal.startswith('~'):
                literals.add(literal[1:])
            else:
                literals.add(literal)

        n = len(literals)
        tseitin_vars = [f'y{i}' for i in range(1, n + 1)]
        clauses = []

        for literal in sat_instance.replace(' or ', '|').replace(' and ', '&').split():
```

```

        if literal.startswith('~'):
            y_index = int(tseitin_vars.index(literal[1:]) + 1)
            clause = f'~{tseitin_vars[y_index - 1]} or {literal}'
        else:
            y_index = int(tseitin_vars.index(literal) + 1)
            clause = f'{tseitin_vars[y_index - 1]} or ~{literal}'
        clauses.append(clause)

    for i in range(n):
        clauses.append(f'{tseitin_vars[i]} or ~y{i}')
        clauses.append(f'~{tseitin_vars[i]} or y{i}')

    return ' and '.join(classes)

def dpll_search_tree_diameter(sat_instance):
    # Simplified DPLL search tree diameter calculation
    # This is a placeholder for the actual implementation
    n = len(sat_instance.split(' or '))
    return math.ceil(math.log2(n))

def algebraic_k_group_rank(tseitin_representation):
    # Placeholder for the actual implementation
    # This is a simplified example where rank is proportional to the number of literals
    n = len(tseitin_representation.replace(' or ', '|').replace(' and ', '&').split())
    return n

n = random.randint(5, 30)
sat_instance = generate_random_sat_instance(n)
tseitin_rep = tseitin_representation(sat_instance)
d_pi = dpll_search_tree_diameter(tseitin_rep)
rank_K_pi = algebraic_k_group_rank(tseitin_rep)

return {
    "metric_name": "d( $\pi$ )",
    "metric_value": d_pi,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": d_pi <= math.log2(rank_K_pi),
    "counterexample": "" if d_pi <= math.log2(rank_K_pi) else f"d( $\pi$ ) = {d_pi}, logrank(K_ $\pi$ ) = "
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(1, 1000000) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(result["conjecture_holds"] for result in results):
        mean_d_pi = sum(result["metric_value"] for result in results) / len(results)
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_d_pi} std=0.0 support_fraction={support_fraction}")
    else:

```

```

first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conje
counterexample = next(result["counterexample"] for result in results if not result["conje
print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b640e24d.py", line 93, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b640e24d.py", line 74, in run_trial
    tseitin_rep = tseitin_representation(sat_instance)
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b640e24d.py", line 50, in tseitin_repre
    y_index = int(tseitin_vars.index(literal) + 1)
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: '(~x1' is not in list

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the conjecture's support or falsification. | next: Debug the test code to ensure it runs successfully and produces the required data for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13964
2	propose	ollama_remote	glm4:latest	0	0	14958
3	preregistration	ollama_remote	glm4:latest	0	0	17386
4	novelty	ollama_remote	glm4:latest	0	0	11383
5	novelty	ollama_remote	glm4:latest	0	0	13574
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18165
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15659
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11898
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12467
10	judge	ollama_remote	glm4:latest	0	0	8924

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 138378 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/c6cc8034e7cc.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/c6cc8034e7cc.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/c6cc8034e7cc.tar.gz (if generated)